

Exploratory Data Analysis of Used-Car Resale Data

Project: Car Resell Prediction & Recommendation System

Run each cell from top to bottom. Export the completed notebook as PDF from Colab (File → Print → Save as PDF).

1. Setup and Dataset Upload

Upload `UsedCars.csv` . The analysis uses the same core conversion and cleaning logic as the project application.

```
In [2]: # Run once in Colab if packages are missing:
# !pip -q install pandas matplotlib seaborn
from google.colab import files
from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style='whitegrid', palette='deep')
uploaded = files.upload()
DATA_PATH = next(iter(uploaded))
df_raw = pd.read_csv(DATA_PATH)
print('Dataset shape:', df_raw.shape)
display(df_raw.head())
```

Choose files No file chosen

Upload widget is only available when the cell has

been executed in the current browser session. Please rerun this cell to enable.

Saving UsedCars.csv to UsedCars.csv

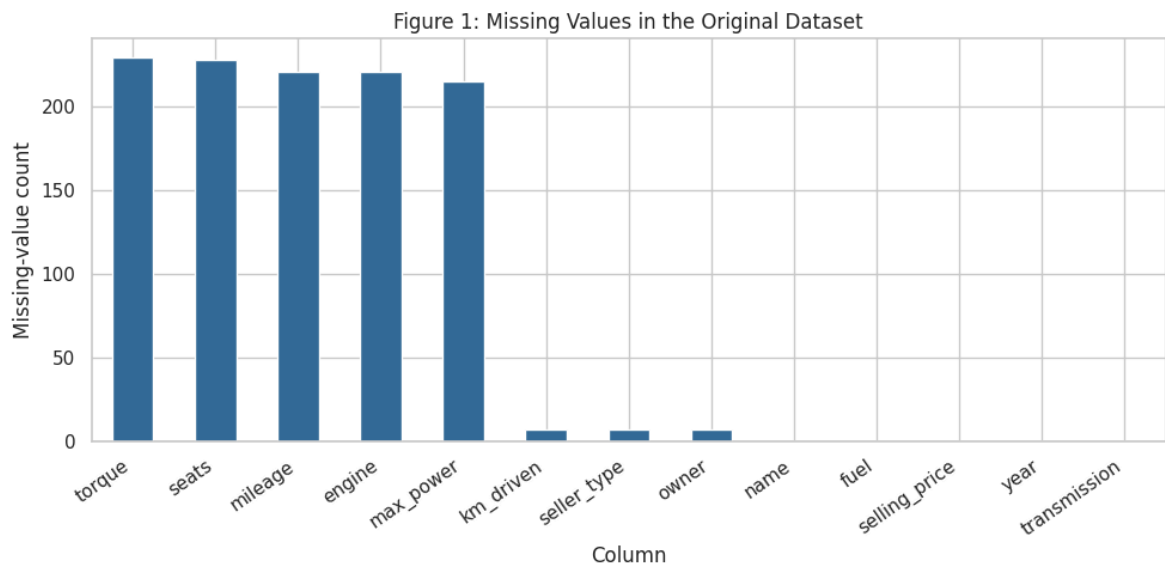
Dataset shape: (8135, 13)

	name	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mi
0	Maruti Swift Dzire VDI	2014	450000.0	145500.0	Diesel	Individual	Manual	First Owner	
1	Skoda Rapid 1.5 TDI Ambition	2014	370000.0	120000.0	Diesel	Individual	Manual	Second Owner	
2	Honda City 2017-2020 EXi	2006	158000.0	140000.0	Petrol	Individual	Manual	Third Owner	
3	Hyundai i20 Sportz Diesel	2010	225000.0	127000.0	Diesel	Individual	Manual	First Owner	
4	Maruti Swift VXi BSIII	2007	130000.0	120000.0	Petrol	Individual	Manual	First Owner	

2. Data Quality: Missing Values

This figure identifies variables that need attention before model training. Text columns and technical fields may contain missing values, so numeric values will be converted carefully and later handled by the model pipeline.

```
In [3]: missing = df_raw.isna().sum().sort_values(ascending=False)
fig, ax = plt.subplots(figsize=(10, 5))
missing.plot(kind='bar', ax=ax, color='#356b9a')
ax.set(title='Figure 1: Missing Values in the Original Dataset', xlabel='Column',
plt.xticks(rotation=35, ha='right'); plt.tight_layout(); plt.show()
print('Duplicate rows:', df_raw.duplicated().sum())
display(missing.to_frame('Missing values'))
```



Duplicate rows: 1202

Missing values	
torque	229
seats	228
mileage	221
engine	221
max_power	215
km_driven	7
seller_type	7
owner	7
name	0
fuel	0
selling_price	0
year	0
transmission	0

Conclusion

The original file contains **1,202 duplicate rows**. The highest missing counts occur in `torque` (229), `seats` (228), `mileage` (221), `engine` (221), and `max_power` (215). The project drops `torque`, removes duplicates, converts technical text fields to numeric values, and imputes remaining numeric missing values in the prediction pipeline.

3. Cleaning and Feature Preparation

The code below converts the target price to the project NPR representation, removes duplicates, extracts brand/model, converts technical specifications to numbers, and trims

the bottom and top 1% of selling prices.

```
In [4]: df = df_raw.copy()
df['selling_price'] = df['selling_price'] * 2.2
df = df.drop(columns=['torque'], errors='ignore').drop_duplicates()
df['brand'] = df['name'].astype(str).str.split().str[0]
df['model'] = df['name'].astype(str).str.split().str[1:].str.join(' ')
for col, suffix in [('mileage', ' kmpl'), ('engine', ' CC'), ('max_power', ' bhp')]:
    df[col] = pd.to_numeric(df[col].astype(str).str.replace(suffix, '', regex=False))
q01, q99 = df['selling_price'].quantile([0.01, 0.99])
df = df[df['selling_price'].between(q01, q99)].copy()
df['log_selling_price'] = np.log1p(df['selling_price'])
df['car_age'] = pd.Timestamp.now().year - df['year']
df['km_per_year'] = df['km_driven'] / (df['car_age'] + 1)
print('Cleaned shape:', df.shape)
print(f'Price limits after trimming: NPR {q01:,.0f} to NPR {q99:,.0f}')
display(df.head())
```

Cleaned shape: (6793, 17)

Price limits after trimming: NPR 111,408 to NPR 6,022,544

	name	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mi
0	Maruti Swift Dzire VDI	2014	990000.0	145500.0	Diesel	Individual	Manual	First Owner	
1	Skoda Rapid 1.5 TDI Ambition	2014	814000.0	120000.0	Diesel	Individual	Manual	Second Owner	
2	Honda City 2017-2020 EXi	2006	347600.0	140000.0	Petrol	Individual	Manual	Third Owner	
3	Hyundai i20 Sportz Diesel	2010	495000.0	127000.0	Diesel	Individual	Manual	First Owner	
4	Maruti Swift VXi BSIII	2007	286000.0	120000.0	Petrol	Individual	Manual	First Owner	

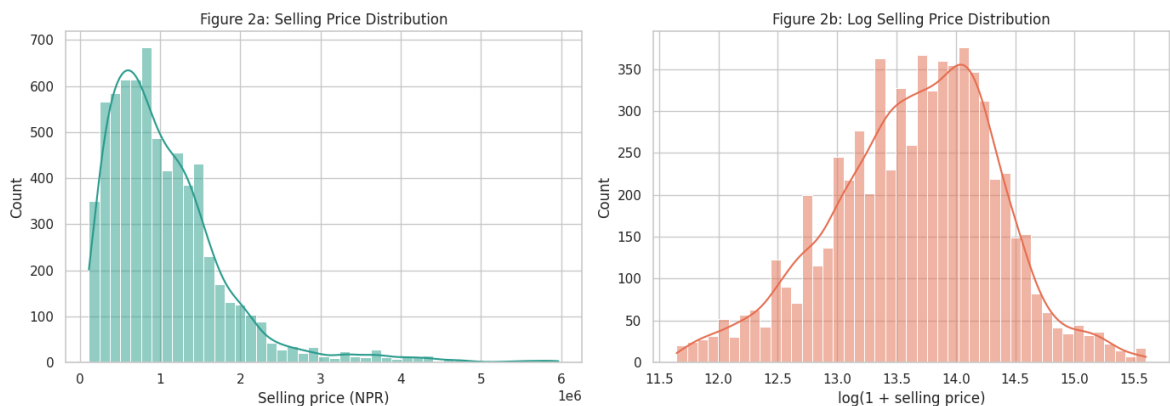
Conclusion

After duplicate removal and 1%-99% price trimming, the analysis dataset contains **6,793 records**. This reduces the influence of repeated records and extreme price values before visual analysis and model training.

4. Selling-Price Distribution

The first plot shows the resale-price distribution after cleaning. The second uses a logarithmic transformation, which is also used by the model target.

```
In [5]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.histplot(df['selling_price'], bins=45, kde=True, ax=axes[0], color='#2a9d8f')
axes[0].set(title='Figure 2a: Selling Price Distribution', xlabel='Selling price (NPR)', ylabel='Count')
sns.histplot(df['log_selling_price'], bins=45, kde=True, ax=axes[1], color='#e76f33')
axes[1].set(title='Figure 2b: Log Selling Price Distribution', xlabel='log(1 + selling price)', ylabel='Count')
plt.tight_layout(); plt.show()
print(f'Median price: NPR {df.selling_price.median():,.0f}')
print(f'Mean price: NPR {df.selling_price.mean():,.0f}')
```



Median price: NPR 880,000

Mean price: NPR 1,068,648

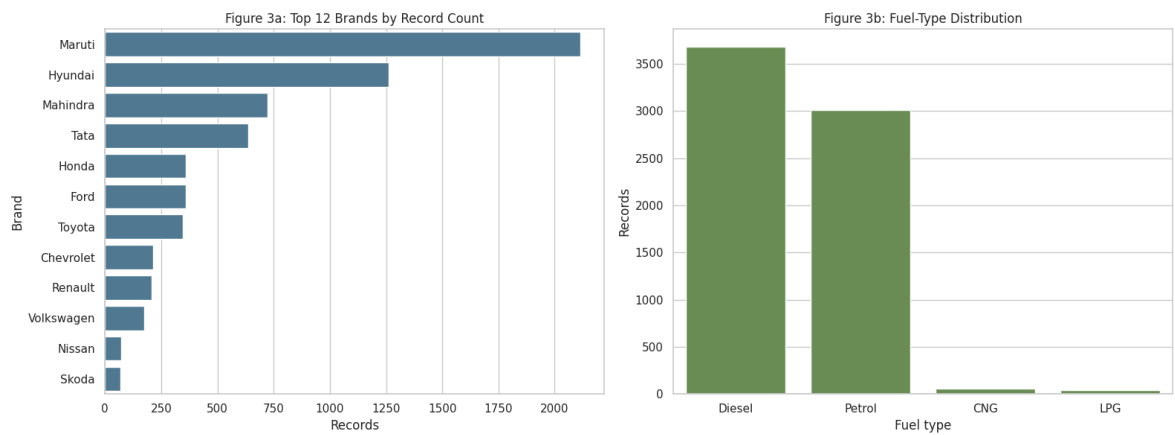
Conclusion

The mean cleaned price is about **NPR 1,068,648**, while the median is about **NPR 880,000**. Because the mean is higher than the median, the original price scale is right-skewed. The log transformation makes the target distribution more suitable for the linear SGD model.

5. Brand and Category Distribution

These plots show whether the dataset is balanced across brands, fuel types, transmission, and ownership groups.

```
In [6]: fig, axes = plt.subplots(1, 2, figsize=(16, 6))
order = df['brand'].value_counts().head(12).index
sns.countplot(data=df, y='brand', order=order, ax=axes[0], color='#457b9d')
axes[0].set(title='Figure 3a: Top 12 Brands by Record Count', xlabel='Records', ylabel='Count')
sns.countplot(data=df, x='fuel', order=df['fuel'].value_counts().index, ax=axes[1], color='#457b9d')
axes[1].set(title='Figure 3b: Fuel-Type Distribution', xlabel='Fuel type', ylabel='Count')
plt.tight_layout(); plt.show()
display(pd.crosstab(df['fuel'], df['transmission']))
```



transmission	Automatic	Manual
fuel		
CNG	0	57
Diesel	240	3446
LPG	0	38
Petrol	280	2732

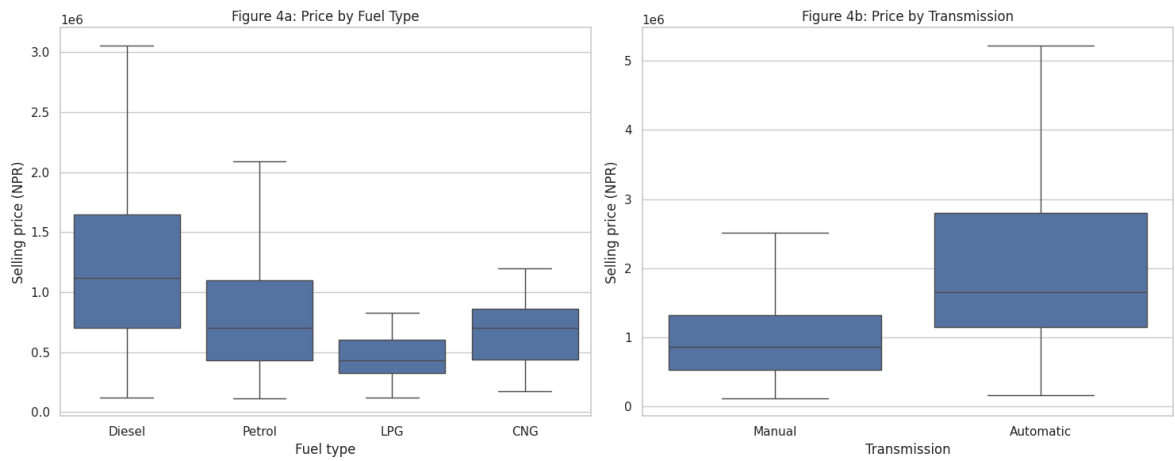
Conclusion

The dataset is dominated by **Maruti (2,114)** and **Hyundai (1,262)** records. Diesel (3,686) and petrol (3,012) are the most frequent fuel types. Manual transmission is much more common (6,273 records) than automatic transmission (520 records). This imbalance means predictions for rare brands, fuel types, and automatic vehicles may be less reliable.

6. Price Variation by Vehicle Category

Box plots compare the distribution of selling prices between fuel types and transmission types while hiding extreme points for readability.

```
In [7]: fig, axes = plt.subplots(1, 2, figsize=(15, 6))
sns.boxplot(data=df, x='fuel', y='selling_price', ax=axes[0], showfliers=False)
axes[0].set(title='Figure 4a: Price by Fuel Type', xlabel='Fuel type', ylabel='Sei
sns.boxplot(data=df, x='transmission', y='selling_price', ax=axes[1], showfliers=f
axes[1].set(title='Figure 4b: Price by Transmission', xlabel='Transmission', ylab
plt.tight_layout(); plt.show()
```



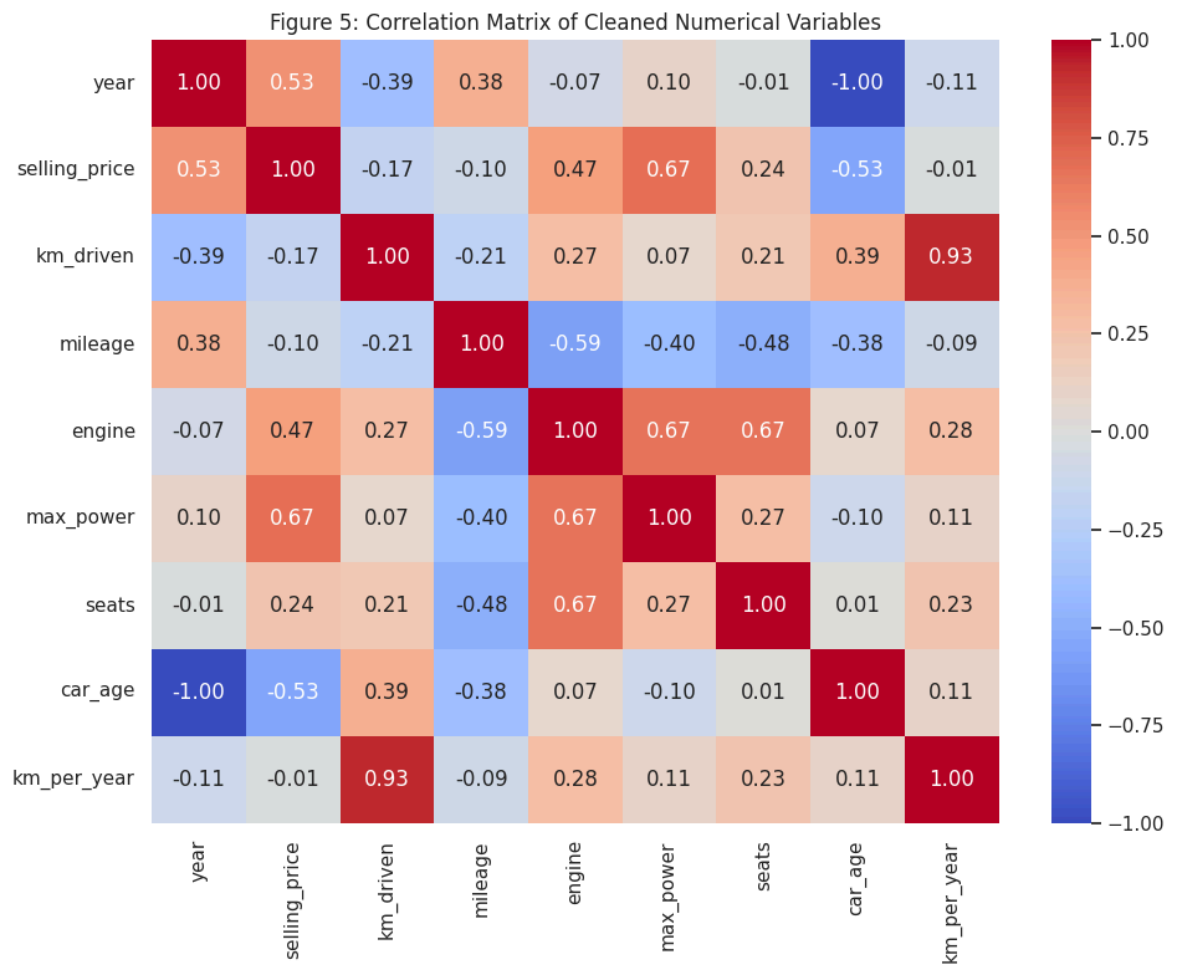
Conclusion

Price distributions differ across fuel and transmission categories. This supports keeping these fields as model inputs instead of estimating price from age and kilometers alone. The box plots should be cited in the report as evidence of category-level price variation.

7. Numerical Correlation Analysis

The correlation matrix measures linear association among cleaned numerical features. Correlation does not prove causation, but it helps identify variables that are useful to investigate in the model.

```
In [10]: cols = ['year', 'selling_price', 'km_driven', 'mileage', 'engine', 'max_power', 's']
corr = df[cols].corr(numeric_only=True)
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0, ax=ax)
ax.set_title('Figure 5: Correlation Matrix of Cleaned Numerical Variables')
plt.tight_layout(); plt.show()
```



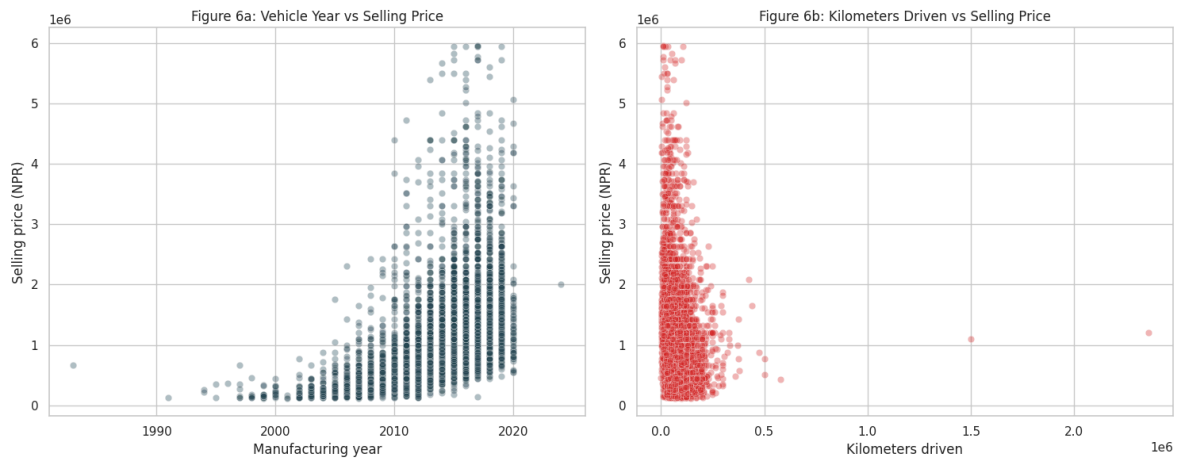
Conclusion

Maximum power has the strongest positive linear correlation with price (about **0.67**), followed by year (about **0.53**) and engine capacity (about **0.47**). Kilometers driven has a negative correlation (about **-0.17**). This supports the use of technical specifications, vehicle age, and usage features in the prediction model.

8. Key Feature-to-Price Relationships

Scatter plots make the relationships between resale price, manufacturing year, and kilometers driven easier to interpret.

```
In [9]: fig, axes = plt.subplots(1, 2, figsize=(15, 6))
sns.scatterplot(data=df, x='year', y='selling_price', alpha=.35, ax=axes[0], color='red')
axes[0].set(title='Figure 6a: Vehicle Year vs Selling Price', xlabel='Manufacturing Year')
sns.scatterplot(data=df, x='km_driven', y='selling_price', alpha=.35, ax=axes[1], color='blue')
axes[1].set(title='Figure 6b: Kilometers Driven vs Selling Price', xlabel='Kilometers Driven')
plt.tight_layout(); plt.show()
```

Final EDA Conclusion

The EDA confirms that resale price is not determined by a single variable. Newer vehicles, more powerful engines, and higher engine capacity generally associate with higher prices, while higher kilometers driven tends to associate with lower prices. The dataset also has missing values, duplicates, outliers, and strong category imbalance. Therefore, cleaning, numeric conversion, one-hot encoding, scaling, price transformation, and careful validation are necessary before training the resale-price prediction model.

In [9]: